

Logistic Regression with Regularization Penalties and Decision Trees

A Comparative Theory Applied to the Titanic Dataset

Daniel Siretanu

August 15, 2026

Short Summary

We present a comparative study of parametric linear models and non-parametric tree architectures applied to the Kaggle Titanic dataset ($n_{\text{train}} = 891$, $n_{\text{test}} = 418$). We evaluate six configurations: unregularized, Ridge (L_2), Lasso (L_1), and Elastic Net logistic regression, alongside a single decision tree and a Random Forest. For logistic regression, we derive the cross-entropy loss from Bernoulli maximum likelihood and formulate the weight update rules across L_1 , L_2 , and Elastic Net penalties. For tree models, we prove that making splits under the impurity criterion never increases expected impurity and formalize the recursive decision rules that govern partitioning. Empirical evaluation on 418 unseen test passengers reveals high agreement (97.85%) between the Random Forest ensemble and Unregularized Logistic Regression, with Random Forest achieving a top Kaggle score of 0.77751, closely followed by Unregularized Logistic Regression at 0.77511.

Contents

1	Introduction	3
1.1	The Motivating Problem	3
2	Logistic Regression as a Generalized Linear Model	3
2.1	The Linear Predictor and the Sigmoid	3
2.2	Maximum Likelihood and the Cross Entropy Loss	4
2.3	The Gradient of the Cross Entropy Loss	5
2.4	Convexity of the Cross Entropy Loss	7
2.5	L_2 , L_1 , and Elastic Net Regularization	8
2.5.1	L_2 (Ridge) Regularization and Weight Decay	9
2.5.2	L_1 (Lasso) Regularization and Feature Sparsity	10
2.5.3	Elastic Net Regularization	10
2.5.4	Comparison and Model Selection	11
3	Tree Based Ensemble Methods	12
3.1	A Single Decision Tree	12
3.2	Random Forest: Averaging Decorrelated Trees	14
4	Model Comparison: Decision Trees, Random Forests, and Logistic Regression	16
5	Empirical Validation on Real Data	17
	Appendices	19

A Appendix: Derivation of L_1 Weight Updates	19
B Appendix: Derivation of Elastic Net Updates and Behavior of Correlated Features	22

1 Introduction

1.1 The Motivating Problem

Given passenger ticket class, sex, age, family size, fare, and a set of engineered features, our goal is to predict binary survival in the historical sinking of the *Titanic*. This binary classification task originates from the standard [Kaggle](#) dataset, providing $n_{\text{train}} = 891$ labeled training observations alongside an unseen evaluation set of $n_{\text{test}} = 418$ passengers. We evaluate six distinct model configurations belonging to two different approaches.

Family	Models
Linear	Logistic Regression (No Penalty, L_1 , L_2 , Elastic Net)
Tree based	Decision Tree, Random Forest

For logistic regression, we derive the sigmoid function, Bernoulli cross entropy loss, and gradients from first principles, proving loss convexity (Theorem 2.3) and examining L_1 , L_2 , and Elastic Net regularization. For tree based models, we formalize node impurity and Information Gain to prove that splitting never increases expected impurity (Proposition 3.1), while showing how Random Forests reduce variance via bootstrap sampling (Lemma 3.1) and feature selection ($m_{\text{try}} = \lfloor \sqrt{p} \rfloor$).

We evaluate both approaches on Kaggle’s test set of 418 unseen passengers to test model generalization.

2 Logistic Regression as a Generalized Linear Model

The first approach we choose for predicting passenger survival on the Titanic is **logistic regression**. This section establishes the mathematical foundations of the model: from parameterization and maximum likelihood estimation to convex optimization and regularization.

2.1 The Linear Predictor and the Sigmoid

The training set consists of n passengers, $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, where:

- $y_i \in \{0, 1\}$ records whether passenger i survived ($y_i = 1$) or did not ($y_i = 0$).
- $x_i \in \mathbb{R}^p$ collects their features: ticket class, age, fare paid, family size, and engineered attributes.

Stacking these gives $X = (x_1, \dots, x_n)^\top \in \mathbb{R}^{n \times p}$ and $y = (y_1, \dots, y_n)^\top \in \{0, 1\}^n$.

A weight vector $w \in \mathbb{R}^p$ and an intercept $b \in \mathbb{R}$ combine a passenger’s features into the linear predictor z_i :

$$z_i := w^\top x_i + b, \quad i = 1, \dots, n.$$

Here, $z_i \in \mathbb{R}$ represents the linear score or log odds of survival for passenger i . The weight w_j measures the change in score per unit change in feature j holding other inputs fixed, while b serves as a baseline offset shared across all passengers.

To map this unbounded score z_i into a valid survival probability, we assume:

$$y_i | x_i \sim \text{Bernoulli}(p(x_i)), \quad p(x_i) := P(y_i = 1 | x_i) \in (0, 1), \quad \text{independently across } i.$$

Because $p(x_i) \in (0, 1)$ while the score z_i ranges over $\mathbb{R}$, the two are related through the logit function:¹

$$\text{logit}(p(x_i)) := \log \frac{p(x_i)}{1 - p(x_i)} = z_i. \quad (1)$$

¹The logit function maps the constrained probability interval $(0, 1)$ monotonically onto the real line $\mathbb{R}$, providing a smooth function.

Proposition 2.1 (Sigmoid)

Equation (1) solved for $p(x_i)$ gives $p(x_i) = \sigma(z_i)$, where $\sigma(z) := 1/(1 + e^{-z})$.

Proof.

Taking the exponential of both sides of Equation (1) gives:

$$\frac{p}{1-p} = e^z \implies p = e^z(1-p) = e^z - pe^z.$$

Rearranging terms to isolate p :

$$p(1 + e^z) = e^z \implies p = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z). \quad \square$$

Proposition 2.1 establishes that $p(x_i) = \sigma(z_i)$: the true survival probability equals the sigmoid of the linear predictor. We name this quantity

$$\hat{y}_i := \sigma(z_i) = \sigma(w^\top x_i + b) \quad (2)$$

the **model's predicted survival probability**. By definition, $\hat{y}_i$ is computed from (w, b) and x_i alone. Substituting $\hat{y}_i$ for $p(x_i)$ in the assumption above yields the model used throughout this work:

$$y_i \mid x_i \sim \text{Bernoulli}(\hat{y}_i), \quad \text{independently across } i. \quad (3)$$

Lemma 2.1 (Sigmoid derivative)

A useful result for subsequent calculations (Theorem 2.2) and Hessian (Lemma 2.2) is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Proof.

Differentiating $\sigma(z) = (1 + e^{-z})^{-1}$ via the quotient or chain rule yields:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}.$$

Noting that $1 - \sigma(z) = 1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}$, we obtain:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad \square$$

2.2 Maximum Likelihood and the Cross Entropy Loss

Equation (2) defines $\hat{y}_i = \sigma(w^\top x_i + b)$ for an arbitrary choice of (w, b) . It does not yet tell us which choice to make. We select (w, b) through the principle of maximum likelihood.

Definition 2.1 (Likelihood)

Given the model $y_i \mid x_i \sim \text{Bernoulli}(\hat{y}_i)$ with $\hat{y}_i = \sigma(w^\top x_i + b)$ independently across i , the *likelihood* of (w, b) is the probability assigned to the observed binary outcomes $y_1, \dots, y_n$ as a function of the parameters:

$$L(w, b) := \prod_{i=1}^n P(y_i \mid x_i) = \prod_{i=1}^n \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}.$$

The observed data (x_i, y_i) remain fixed throughout. Only (w, b) vary, entering through $\hat{y}_i$. The *maximum likelihood estimate (MLE)* is defined as:

$$(\hat{w}, \hat{b}) := \underset{w, b}{\operatorname{argmax}} L(w, b),$$

which identifies the parameters under which the observed survival outcomes were most probable.

Theorem 2.1 (Cross entropy is the negative log likelihood)

The MLE (Definition 2.1) is equivalent to finding parameters (w, b) that minimize the empirical **cross entropy loss**:²

$$\text{CE}(w, b) := -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

Proof.

By Definition 2.1, $L(w, b) = \prod_{i=1}^n \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$. Because the natural logarithm is strictly increasing, maximizing $L(w, b)$ gives the same optimal parameters as maximizing its log likelihood:

$$\log L(w, b) = \sum_{i=1}^n \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

Maximizing $\log L(w, b)$ is equivalent to minimizing its negative, $-\log L(w, b)$. Dividing by the size of the dataset n does not change the optimal parameters, but converts the total loss into an average loss per sample:

$$\underset{w, b}{\operatorname{argmax}} L(w, b) = \underset{w, b}{\operatorname{argmin}} \left(-\frac{1}{n} \log L(w, b) \right) = \underset{w, b}{\operatorname{argmin}} \text{CE}(w, b). \quad \square$$

2.3 The Gradient of the Cross Entropy Loss

By Theorem 2.1, fitting the model corresponds to minimizing $\text{CE}(w, b)$. We achieve this via gradient descent: starting from an initial parameter configuration (w, b) , we iterate step by step in the direction of steepest decrease, opposite to ∇CE .³

To tidy up the notation, we combine w and b into a single parameter vector $\theta := (b, w^\top)^\top \in \mathbb{R}^{p+1}$ and augment the features matrix with a leading column of ones: $\tilde{X} := [\mathbf{1} \mid X] \in \mathbb{R}^{n \times (p+1)}$. Under this parameterization, the linear predictor for passenger i is:

$$z_i = \tilde{x}_i^\top \theta, \quad \text{where } \tilde{x}_i := (1, x_i^\top)^\top \in \mathbb{R}^{p+1}.$$

Theorem 2.2 (Gradient of the cross entropy loss)

The gradient of the cross entropy loss with respect to θ is:

$$\nabla_\theta \text{CE}(\theta) = \frac{1}{n} \tilde{X}^\top (\hat{y} - y).$$

Proof.

Express $\text{CE}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_i$, where $\ell_i := -[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$ denotes the entropy loss contribution of passenger i . Differentiating ℓ_i with respect to θ via the chain rule through $\theta \rightarrow z_i \rightarrow \hat{y}_i \rightarrow \ell_i$:

$$\frac{\partial \ell_i}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i} + \frac{1 - y_i}{1 - \hat{y}_i} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}.$$

²The term cross entropy comes from information theory and statistical physics, where it measures the difference between the true distribution of data and our model's predictions. Minimizing cross entropy forces predicted probabilities to match observed outcomes as closely as possible.

³For a scalar loss function, the gradient $\nabla_\theta \text{CE}(\theta)$ is simply a column vector of length $p + 1$ containing the partial derivative for each parameter:

$$\nabla_\theta \text{CE}(\theta) = \begin{pmatrix} \frac{\partial \text{CE}}{\partial b} \\ \frac{\partial \text{CE}}{\partial w_1} \\ \vdots \\ \frac{\partial \text{CE}}{\partial w_p} \end{pmatrix} \in \mathbb{R}^{p+1}.$$

By Lemma 2.1, $\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$. Combining these yields:

$$\frac{\partial \ell_i}{\partial z_i} = \frac{\partial \ell_i}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \cdot \hat{y}_i(1 - \hat{y}_i) = \hat{y}_i - y_i.$$

Since $\frac{\partial z_i}{\partial \theta} = \tilde{x}_i$, the gradient for individual i is $\frac{\partial \ell_i}{\partial \theta} = (\hat{y}_i - y_i)\tilde{x}_i$. Averaging across all n passengers:

$$\nabla_{\theta} \text{CE}(\theta) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)\tilde{x}_i = \frac{1}{n} \tilde{X}^{\top}(\hat{y} - y). \quad \square$$

Corollary 2.1 (The gradient in (w, b) coordinates)

Writing $\theta = (b, w^{\top})^{\top}$ and $\tilde{X} = [\mathbf{1} \mid X]$, the gradient system in Theorem 2.2 splits into separate partial derivatives:

$$\frac{\partial \text{CE}}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i), \quad \frac{\partial \text{CE}}{\partial w} = \frac{1}{n} X^{\top}(\hat{y} - y).$$

Proof.

Writing $\tilde{X} = [\mathbf{1} \mid X]$ in block matrix form gives $\tilde{X}^{\top} = \begin{pmatrix} \mathbf{1}^{\top} \\ X^{\top} \end{pmatrix}$. Matrix vector multiplication yields:

$$\tilde{X}^{\top}(\hat{y} - y) = \begin{pmatrix} \mathbf{1}^{\top}(\hat{y} - y) \\ X^{\top}(\hat{y} - y) \end{pmatrix}.$$

Since $\theta = (b, w^{\top})^{\top}$, scaling by $1/n$ separates the gradient into its partial derivative:

$$\begin{aligned} \frac{\partial \text{CE}}{\partial b} &= \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \\ \frac{\partial \text{CE}}{\partial w} &= \frac{1}{n} X^{\top}(\hat{y} - y) \end{aligned}$$

□

Remark 2.1 (Absence of a closed form solution)

Setting the gradient in Theorem 2.2 to zero yields the stationary condition:

$$\tilde{X}^{\top}(\hat{y} - y) = 0, \quad \text{where } \hat{y} = \sigma(\tilde{X}\theta).$$

In standard linear regression, the corresponding condition is $\tilde{X}^{\top}(\tilde{X}\theta - y) = 0$, which simplifies to the normal equations $\tilde{X}^{\top}\tilde{X}\theta = \tilde{X}^{\top}y$ and admits the analytical closed form solution $\theta = (\tilde{X}^{\top}\tilde{X})^{-1}\tilde{X}^{\top}y$. For logistic regression, however, $\hat{y} = \sigma(\tilde{X}\theta)$ is non-linear, meaning parameter estimation requires iterative numerical optimization.

Definition 2.2 (Gradient descent)

Starting from an initial parameter vector $\theta^{(0)}$ (e.g., $\theta^{(0)} = \mathbf{0}$), gradient descent constructs a sequence of parameter updates:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \nabla_{\theta} \text{CE}(\theta^{(t)}), \quad t = 0, 1, 2, \dots, \quad (4)$$

given a positive learning rate $\alpha > 0$, running until a maximum number of iterations is reached.⁴ Expressed in (w, b) coordinates via Corollary 2.1, the updates take the form:

$$w^{(t+1)} := w^{(t)} - \frac{\alpha}{n} X^{\top}(\hat{y}^{(t)} - y), \quad b^{(t+1)} := b^{(t)} - \frac{\alpha}{n} \sum_{i=1}^n (\hat{y}_i^{(t)} - y_i). \quad (5)$$

⁴Using the first order Taylor expansion $\text{CE}(\theta^{(t)} + \Delta\theta) \approx \text{CE}(\theta^{(t)}) + \nabla_{\theta} \text{CE}(\theta^{(t)})^{\top} \Delta\theta$, minimizing the updated loss requires choosing $\Delta\theta$ to make the inner product $\nabla_{\theta} \text{CE}(\theta^{(t)})^{\top} \Delta\theta$ as negative as possible. By the Cauchy Schwarz inequality, $|\nabla_{\theta} \text{CE}(\theta^{(t)})^{\top} \Delta\theta| \leq \|\nabla_{\theta} \text{CE}(\theta^{(t)})\|_2 \|\Delta\theta\|_2$, which implies $\nabla_{\theta} \text{CE}(\theta^{(t)})^{\top} \Delta\theta \geq -\|\nabla_{\theta} \text{CE}(\theta^{(t)})\|_2 \|\Delta\theta\|_2$. Equality holds with maximum negative value occurring if and only if $\Delta\theta$ points in the exact opposite direction of the gradient vector, giving $\Delta\theta = -\alpha \nabla_{\theta} \text{CE}(\theta^{(t)})$. Updating the parameters via $\theta^{(t+1)} = \theta^{(t)} + \Delta\theta$ gives equation (4).

2.4 Convexity of the Cross Entropy Loss

In practice, fitting the model requires running gradient descent until the gradient is numerically zero. However, a zero gradient ($\nabla_{\theta}\text{CE}(\theta) = 0$) only indicates that θ is a stationary point, which could represent a global minimum, a local minimum, a local maximum, or a saddle point. Convexity ensures that any stationary point is a global minimum. We verify this property by deriving the Hessian matrix and establishing that it is positive semi-definite everywhere.

Lemma 2.2 (Hessian)

The Hessian of $\text{CE}(\theta)$ is given by:⁵

$$\nabla_{\theta}^2 \text{CE}(\theta) = \frac{1}{n} \tilde{X}^{\top} D \tilde{X}, \quad \text{where } D := \text{diag}(\hat{y}_1(1 - \hat{y}_1), \dots, \hat{y}_n(1 - \hat{y}_n)).$$

Proof.

By definition, the Hessian matrix is obtained by differentiating the column vector gradient $\nabla_{\theta}\text{CE}(\theta) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \tilde{x}_i$ with respect to the row vector $\theta^{\top}$:

$$\nabla_{\theta}^2 \text{CE}(\theta) = \frac{\partial}{\partial \theta^{\top}} \left(\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \tilde{x}_i \right) = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i \frac{\partial (\hat{y}_i - y_i)}{\partial \theta^{\top}}.$$

Because the observation label y_i is a constant scalar, its derivative with respect to $\theta^{\top}$ is zero. Therefore, $\frac{\partial (\hat{y}_i - y_i)}{\partial \theta^{\top}} = \frac{\partial \hat{y}_i}{\partial \theta^{\top}}$. We apply the multivariable chain rule to compute row vector $\frac{\partial \hat{y}_i}{\partial \theta^{\top}}$ through intermediate score $z_i = \theta^{\top} \tilde{x}_i$:

$$\frac{\partial \hat{y}_i}{\partial \theta^{\top}} = \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \theta^{\top}}.$$

We evaluate each factor individually:

1. For the linear predictor $z_i = \theta^{\top} \tilde{x}_i$, differentiating with respect to row vector $\theta^{\top}$ yields the row vector $\frac{\partial z_i}{\partial \theta^{\top}} = \tilde{x}_i^{\top}$.⁶
2. Applying Lemma 2.1 for prediction $\hat{y}_i = \sigma(z_i)$ gives scalar derivative $\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$.

Multiplying scalar $\hat{y}_i(1 - \hat{y}_i)$ by row vector $\tilde{x}_i^{\top}$ gives:

$$\frac{\partial \hat{y}_i}{\partial \theta^{\top}} = \hat{y}_i(1 - \hat{y}_i) \tilde{x}_i^{\top}.$$

Substituting this row vector back into our summation yields:

$$\nabla_{\theta}^2 \text{CE}(\theta) = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i \left(\hat{y}_i(1 - \hat{y}_i) \tilde{x}_i^{\top} \right) = \frac{1}{n} \sum_{i=1}^n \hat{y}_i(1 - \hat{y}_i) \tilde{x}_i \tilde{x}_i^{\top}.$$

⁵The Hessian matrix $\nabla_{\theta}^2 \text{CE}(\theta) \in \mathbb{R}^{(p+1) \times (p+1)}$ is defined as the matrix of second derivatives $\frac{\partial}{\partial \theta^{\top}} \left(\frac{\partial \text{CE}}{\partial \theta} \right)$:

$$\nabla_{\theta}^2 \text{CE}(\theta) = \begin{pmatrix} \frac{\partial^2 \text{CE}}{\partial \theta_0^2} & \frac{\partial^2 \text{CE}}{\partial \theta_0 \partial \theta_1} & \cdots & \frac{\partial^2 \text{CE}}{\partial \theta_0 \partial \theta_p} \\ \frac{\partial^2 \text{CE}}{\partial \theta_1 \partial \theta_0} & \frac{\partial^2 \text{CE}}{\partial \theta_1^2} & \cdots & \frac{\partial^2 \text{CE}}{\partial \theta_1 \partial \theta_p} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 \text{CE}}{\partial \theta_p \partial \theta_0} & \frac{\partial^2 \text{CE}}{\partial \theta_p \partial \theta_1} & \cdots & \frac{\partial^2 \text{CE}}{\partial \theta_p^2} \end{pmatrix}.$$

Differentiating the column gradient vector $\frac{\partial \text{CE}}{\partial \theta} \in \mathbb{R}^{(p+1) \times 1}$ with respect to the transposed row vector $\theta^{\top} \in \mathbb{R}^{1 \times (p+1)}$ yields all second order partial derivatives $H_{jk} = \frac{\partial^2 \text{CE}}{\partial \theta_j \partial \theta_k}$.

⁶Because $z_i = \theta^{\top} \tilde{x}_i = \sum_{j=0}^p \theta_j \tilde{x}_{ij}$ is a scalar function of parameter vector θ , differentiating z_i entry by entry with respect to row vector $\theta^{\top} = (\theta_0 \quad \theta_1 \quad \dots \quad \theta_p)$ yields $\frac{\partial z_i}{\partial \theta^{\top}} = \begin{pmatrix} \frac{\partial z_i}{\partial \theta_0} & \frac{\partial z_i}{\partial \theta_1} & \cdots & \frac{\partial z_i}{\partial \theta_p} \end{pmatrix} = (\tilde{x}_{i0} \quad \tilde{x}_{i1} \quad \dots \quad \tilde{x}_{ip}) = \tilde{x}_i^{\top}$.

We note that $\tilde{x}_i \tilde{x}_i^\top$ is the outer product of column vector $\tilde{x}_i$ with row vector $\tilde{x}_i^\top$, producing a $(p+1) \times (p+1)$ matrix. Recall that $\tilde{X} \in \mathbb{R}^{n \times (p+1)}$ is the feature matrix augmented with a leading column of ones, whose i -th row is $\tilde{x}_i^\top$. Defining the diagonal matrix $D = \text{diag}(\hat{y}_1(1 - \hat{y}_1), \dots, \hat{y}_n(1 - \hat{y}_n))$ and expanding the matrix product $\tilde{X}^\top D \tilde{X}$ yields a weighted sum of outer products:

$$\frac{1}{n} \tilde{X}^\top D \tilde{X} = \frac{1}{n} \sum_{i=1}^n D_{ii} \tilde{x}_i \tilde{x}_i^\top = \frac{1}{n} \sum_{i=1}^n \hat{y}_i(1 - \hat{y}_i) \tilde{x}_i \tilde{x}_i^\top,$$

which matches the outer product sum derived above. $\square$

Theorem 2.3 (Convexity)

$\text{CE}(\theta)$ is convex on $\mathbb{R}^{p+1}$.

Proof.

A twice differentiable function is convex on $\mathbb{R}^{p+1}$ if and only if its Hessian matrix is positive semi-definite at every point θ , meaning $v^\top \nabla_\theta^2 \text{CE}(\theta) v \geq 0$ for all vectors $v \in \mathbb{R}^{p+1}$.

Using Lemma 2.2, we substitute $\nabla_\theta^2 \text{CE}(\theta) = \frac{1}{n} \tilde{X}^\top D \tilde{X}$ to obtain:

$$v^\top \left(\frac{1}{n} \tilde{X}^\top D \tilde{X} \right) v = \frac{1}{n} (\tilde{X}v)^\top D (\tilde{X}v).$$

Let $u = \tilde{X}v \in \mathbb{R}^n$ be the n -dimensional vector with entries $u_i = (\tilde{X}v)_i = \tilde{x}_i^\top v$. Because D is diagonal, the quadratic form $u^\top D u$ expands directly into a weighted sum of squares:

$$\frac{1}{n} u^\top D u = \frac{1}{n} \sum_{i=1}^n D_{ii} u_i^2 = \frac{1}{n} \sum_{i=1}^n \hat{y}_i(1 - \hat{y}_i) (\tilde{x}_i^\top v)^2.$$

We evaluate the sign of every term in this summation:

1. The squared scalar term satisfies $(\tilde{x}_i^\top v)^2 \geq 0$ for all vectors v .
2. Because the sigmoid function outputs probabilities $\hat{y}_i \in (0, 1)$, we have $\hat{y}_i > 0$ and $1 - \hat{y}_i > 0$, making $\hat{y}_i(1 - \hat{y}_i) > 0$ strictly positive for all i .

Since every term in the sum is a product of non-negative quantities, the total sum is non-negative:

$$v^\top \nabla_\theta^2 \text{CE}(\theta) v = \frac{1}{n} \sum_{i=1}^n \hat{y}_i(1 - \hat{y}_i) (\tilde{x}_i^\top v)^2 \geq 0, \quad \forall v \in \mathbb{R}^{p+1}, \forall \theta \in \mathbb{R}^{p+1}.$$

Thus, $\nabla_\theta^2 \text{CE}(\theta) \succeq 0$ ⁷ everywhere, establishing that $\text{CE}(\theta)$ is convex. $\square$

2.5 L2, L1, and Elastic Net Regularization

By Theorem 2.1, fitting (w, b) minimizes $\text{CE}(w, b)$ directly. Without a penalty on the magnitude of the parameters, optimization drives the model to fit the training data as closely as possible, capturing random sample noise rather than true population trends. Unusually large parameter values may signal overfitting. For instance, a weight such as $w_{\text{Age}} = 0.3$ reflects a plausible effect size, whereas $w_{\text{Age}} = 8.7$ suggests that the model may rely excessively on age due to possible training sample noise.

Regularization prevents overfitting by adding a penalty term to the objective function, punishing large weight magnitudes. Three standard penalties are used in practice:

⁷The notation $A \succeq 0$ simply means that matrix A is positive semi-definite, which ensures that $v^\top A v \geq 0$ for every vector v .

- **L2 (Ridge) penalty:**⁸ $\frac{\lambda}{2n}\|w\|_2^2$, which shrinks weights smoothly toward zero
- **L1 (Lasso) penalty:**⁹ $\frac{\lambda}{n}\|w\|_1$, which enforces sparsity by driving weights to exactly zero
- **Elastic Net penalty:**¹⁰ $\frac{\lambda_1}{n}\|w\|_1 + \frac{\lambda_2}{2n}\|w\|_2^2$, which balances feature selection and weight stabilization

2.5.1 L2 (Ridge) Regularization and Weight Decay

For L2 regularization, the regularized cross-entropy loss function is defined as:

$$J_\lambda(\theta) := \text{CE}(\theta) + \underbrace{\frac{\lambda}{2n}\|w\|_2^2}_{\text{L2 penalty}}, \quad \lambda \geq 0.$$

Optimizing J_λ via gradient descent requires updating the gradient computations from Corollary 2.1.

Corollary 2.2 (Regularized gradient)

The partial derivatives of the L2-regularized objective $J_\lambda(\theta)$ with respect to w and b are:

$$\nabla_w J_\lambda = \underbrace{\frac{1}{n}X^\top(\hat{y} - y)}_{\text{cross-entropy term}} + \underbrace{\frac{\lambda}{n}w}_{\text{L2 penalty term}}, \quad \frac{\partial J_\lambda}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i).$$

Proof.

Because J_λ is additive, its gradient is the sum of the unregularized cross-entropy gradient (Corollary 2.1) and the derivative of the penalty term. Differentiating $\frac{\lambda}{2n}\|w\|_2^2 = \frac{\lambda}{2n} \sum_{j=1}^p w_j^2$ with respect to w_j yields $\frac{\lambda}{n}w_j$. Since the penalty contains no b term, $\partial J_\lambda / \partial b$ remains unchanged. $\square$

Remark 2.2 (Proportional penalty magnitude)

The L2 penalty term $\frac{\lambda}{n}w_j$ scales linearly with w_j : at $w_j = 5.0$, the shrinking force is 5 times greater than at $w_j = 1.0$, and 50 times greater than at $w_j = 0.1$. As $w_j \rightarrow 0$, the penalty term vanishes.

Proposition 2.2 (Weight decay)

Applying gradient descent (Definition 2.2) to J_λ with learning rate α yields the weight update step:

$$w^{(t+1)} := w^{(t)} \left(1 - \frac{\alpha\lambda}{n} \right) - \frac{\alpha}{n} X^\top(\hat{y}^{(t)} - y),$$

while the bias parameter b updates identically to the unregularized case.

Proof.

Substituting Corollary 2.2's gradient into $w^{(t+1)} := w^{(t)} - \alpha \nabla_w J_\lambda$:

$$w^{(t+1)} = w^{(t)} - \alpha \left(\frac{1}{n} X^\top(\hat{y}^{(t)} - y) + \frac{\lambda}{n} w^{(t)} \right) = w^{(t)} \left(1 - \frac{\alpha\lambda}{n} \right) - \frac{\alpha}{n} X^\top(\hat{y}^{(t)} - y). \quad \square$$

⁸The L_2 norm $\|w\|_2 = \sqrt{\sum_{j=1}^p w_j^2}$ represents the Euclidean length of the weight vector. Squaring it yields $\|w\|_2^2 = \sum_{j=1}^p w_j^2 = w^\top w$, which penalizes larger weights quadratically while keeping the function differentiable everywhere.

⁹The L_1 norm $\|w\|_1 = \sum_{j=1}^p |w_j|$ sums the absolute values of the weights. The non-differentiable corner at $w_j = 0$ creates a constant penalization regardless of parameter magnitude, forcing less important feature weights to become exactly zero.

¹⁰The Elastic Net combines both L_1 and L_2 norms to simultaneously.

The term b is left unregularized because w controls feature sensitivity, while b represents the baseline log-odds.¹¹

2.5.2 L1 (Lasso) Regularization and Feature Sparsity

For L1 regularization, the loss function incorporates the absolute sum of weights:

$$J_\lambda(\theta) := \text{CE}(\theta) + \underbrace{\frac{\lambda}{n} \|w\|_1}_{\text{L1 penalty}}, \quad \lambda \geq 0.$$

Because $\|w\|_1 = \sum_{j=1}^p |w_j|$ is non-differentiable at $w_j = 0$, standard gradient descent cannot evaluate the derivative at zero. For non-zero weights ($w_j \neq 0$), the gradient contains the constant penalty contribution $\frac{\lambda}{n} \text{sign}(w_j)$.

To update weights where the penalty is non-differentiable, we use a local quadratic approximation around $w_j^{(t)}$ (see Appendix A for the step-by-step derivation), giving the closed-form weight update:

$$w_j^{(t+1)} = \text{sign}\left(z_j^{(t)}\right) \max\left(0, \left|z_j^{(t)}\right| - \frac{\alpha\lambda}{n}\right), \quad (6)$$

where $z_j^{(t)} = w_j^{(t)} - \frac{\alpha}{n} [X^\top(\hat{y}^{(t)} - y)]_j$ is the unregularized step ($\lambda = 0$). If the unregularized step magnitude $|z_j^{(t)}|$ is less than or equal to the threshold $\frac{\alpha\lambda}{n}$, the weight is driven to exactly zero, removing that feature from the model.

2.5.3 Elastic Net Regularization

Remark 2.3 (Elastic cross-entropy loss and update rule)

Combining both penalties yields the Elastic Net loss formulation:

$$J_{\lambda_1, \lambda_2}(\theta) = \text{CE}(\theta) + \frac{\lambda_1}{n} \|w\|_1 + \frac{\lambda_2}{2n} \|w\|_2^2.$$

This approach offers the complementary strengths of both regularization methods. The L_1 penalty performs feature selection by driving irrelevant weights to zero, while the L_2 penalty stabilizes the model when features are strongly related to each other (for full mathematical derivations and behavior under feature correlation, see Appendix B).

Following the L_1 derivation (Appendix A), we update the following update rule:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2\alpha} \left(w - z_j^{(t)}\right)^2 + \frac{\lambda_1}{n} |w| + \frac{\lambda_2}{2n} w^2 \right\},$$

where $z_j^{(t)} = w_j^{(t)} - \frac{\alpha}{n} [X^\top(\hat{y}^{(t)} - y)]_j$ is the unregularized gradient step. Combining the quadratic terms gives:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2} \left(1 + \frac{\alpha\lambda_2}{n}\right) w^2 - z_j^{(t)} w + \frac{\alpha\lambda_1}{n} |w| \right\}.$$

Solving across non-zero regimes yields the closed-form parameter update:

$$w_j^{(t+1)} = \frac{\text{sign}\left(z_j^{(t)}\right) \max\left(0, \left|z_j^{(t)}\right| - \frac{\alpha\lambda_1}{n}\right)}{1 + \frac{\alpha\lambda_2}{n}}. \quad (7)$$

The numerator applies L_1 penalty to eliminate irrelevant weights, while the L_2 denominator scales down correlated feature groups jointly. The only trade-off is tuning two hyperparameters (λ_1 and λ_2) instead of one.

¹¹When all feature inputs are zero ($x = \mathbf{0}$), the model predicts $\hat{y} = \sigma(b)$, making $b = \log\left(\frac{\hat{y}}{1-\hat{y}}\right)$ the prior log-odds of the survival probability.

2.5.4 Comparison and Model Selection

Remark 2.4 (Comparing L_1 (Lasso) and L_2 (Ridge) Regularization)

As noted in Remark 2.2, Ridge's gradient contribution $\frac{\lambda}{n}w_j$ weakens as $w_j \rightarrow 0$, vanishing at the origin: weights approach zero asymptotically but rarely reach it exactly. In contrast, Lasso's penalty contribution maintains a constant magnitude λ/n regardless of how small w_j becomes. This constant penalty rate drives non-informative weights directly to zero, performing implicit feature selection.

Property	L2 (Ridge)	L1 (Lasso)
Penalty term	$\frac{\lambda}{2n} \sum_{j=1}^p w_j^2$	$\frac{\lambda}{n} \sum_{j=1}^p w_j $
Gradient contribution	$\frac{\lambda}{n} w_j$	$\frac{\lambda}{n} \text{sign}(w_j) \ (w_j \neq 0)$
Penalty magnitude	Proportional to $ w_j $	Constant (λ/n)
Zeroing behavior	Asymptotic shrinkage toward zero	Exact zeroing (sparse weights)
Feature selection	No	Yes

Remark 2.5 (Model selection criteria)

The choice between regularizers depends on the underlying structure of the features:

- **Ridge** is preferred when most features contribute to the target or when features are strongly correlated.¹²
- **Lasso** is best suited for high-dimensional settings with many irrelevant features, as it sets uninformative weights to zero to perform explicit feature selection and produce an interpretable model.
- **Elastic Net** serves as a balanced approach when working with high-dimensional data containing strongly correlated features, setting irrelevant weights to zero while retaining groups of correlated predictors together.¹³

¹²The L_2 gradient penalty is λw_j , meaning larger weights experience a stronger pulling force toward zero than smaller weights. When two features are correlated, this proportional force equalizes their weights rather than letting a single parameter grow excessively large (see Appendix B).

¹³While pure L_1 regularization randomly selects one feature from a group of identical predictors based on initial conditions, the L_2 component strictly penalizes weight disparities, forcing correlated features to share weights equally rather than dropping all but one (see Appendix B).

3 Tree Based Ensemble Methods

This section covers two models: a single Decision Tree and a Random Forest. Unlike logistic regression (Section 2), neither model uses a linear score $z = w^\top x + b$. Instead, a tree splits the feature space into different regions using simple cutoffs. While a single decision tree balances simplicity and over-fitting, Random Forests average many trees together to reduce reliance on the specific training sample.

3.1 A Single Decision Tree

Definition 3.1 (Node Impurity)

For a set of training points $\mathcal{S}$ at a given node,¹⁴ let

$$\hat{p} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \mathbf{1}[y_i = 1]$$

denote the proportion of survival outcomes ($y_i = 1$) in region $\mathcal{S}$, where $\mathbf{1}[\cdot]$ is an indicator function that equals 1 if the condition holds ($y_i = 1$) and 0 otherwise. We measure the heterogeneity (uncertainty) of $\mathcal{S}$ using one of two standard criteria:

- **Gini Impurity:** $G(\mathcal{S}) = 2\hat{p}(1 - \hat{p})$
- **Entropy:** $H(\mathcal{S}) = -\hat{p} \log_2 \hat{p} - (1 - \hat{p}) \log_2 (1 - \hat{p})$

Both metrics satisfy $G(\mathcal{S}) = H(\mathcal{S}) = 0$ when $\hat{p} \in \{0, 1\}$ (meaning all passengers in that region either survived or died) and reach their maximum at $\hat{p} = \frac{1}{2}$ (where outcomes are split evenly, maximum uncertainty).

Definition 3.2 (Candidate Split and Information Gain)

A candidate split on feature j at threshold τ partitions the node dataset $\mathcal{S}$ into two disjoint child subsets:

$$\mathcal{S}_L = \{i \in \mathcal{S} : x_{ij} \leq \tau\} \quad \text{and} \quad \mathcal{S}_R = \{i \in \mathcal{S} : x_{ij} > \tau\}.$$

Let $n = |\mathcal{S}|$, $n_L = |\mathcal{S}_L|$, and $n_R = |\mathcal{S}_R|$. The expected post-split impurity (also called the *conditional impurity*) $Q(j, \tau)$ is the sample-weighted average of the child node impurities:

$$Q(j, \tau) = \frac{n_L}{n} H(\mathcal{S}_L) + \frac{n_R}{n} H(\mathcal{S}_R).$$

The reduction in impurity achieved by performing this split is defined as the **information gain**:

$$\Delta(j, \tau) := H(\mathcal{S}) - Q(j, \tau).$$

(Note: The identical conditional expression $Q(j, \tau)$ and gain $\Delta(j, \tau)$ apply when evaluating impurity using Gini $G(\mathcal{S})$ instead of Entropy $H(\mathcal{S})$.)

Proposition 3.1 (Splitting never increases expected impurity)

For any candidate split on feature j at threshold τ , the information gain is non-negative:

$$\Delta(j, \tau) \geq 0,$$

holding universally for both entropy H and Gini impurity G .

¹⁴A set of training points $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$ refers to the collection of observations present at a specific node, where each point pairs a feature vector $x_i \in \mathbb{R}^p$ (e.g., passenger class, age, fare) with its observed survival status $y_i \in \{0, 1\}$. At the root node, $\mathcal{S}$ contains the full training set. At deeper nodes, $\mathcal{S}$ represents the conditional subset of passengers that satisfy all split criteria leading to that branch.

Proof.

Let $\hat{p}$, $\hat{p}_L$, and $\hat{p}_R$ be the empirical survival rates in $\mathcal{S}$, $\mathcal{S}_L$, and $\mathcal{S}_R$, respectively. By the law of total probability, the parent node's survival rate is the weighted average of its children's rates:

$$\hat{p} = \frac{n_L}{n} \hat{p}_L + \frac{n_R}{n} \hat{p}_R.$$

Because the entropy function $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ is strictly concave in $p \in (0, 1)$, Jensen's inequality¹⁵ applies directly:

$$H(\mathcal{S}) = H\left(\frac{n_L}{n} \hat{p}_L + \frac{n_R}{n} \hat{p}_R\right) \geq \frac{n_L}{n} H(\hat{p}_L) + \frac{n_R}{n} H(\hat{p}_R) = Q(j, \tau).$$

Subtracting $Q(j, \tau)$ from both sides yields $\Delta(j, \tau) = H(\mathcal{S}) - Q(j, \tau) \geq 0$. The identical derivation holds for Gini impurity $G(p) = 2p(1-p)$, which is also strictly concave. $\square$

Proposition 3.1 is what makes tree construction well posed: at every node there is a well defined, non-negative quantity to search over, $(j^*, \tau^*) = \operatorname{argmax}_{j, \tau} \Delta(j, \tau)$, and the search terminates because impurity is monotonically driven toward zero as splits accumulate.

Definition 3.3 (Recursive Tree Construction and Leaf Prediction)

Starting at the root node initialized with the full training set $\mathcal{S}_{\text{root}} = \mathcal{S}$, a decision tree $T(x)$ is constructed recursively as follows:

1. **Evaluate Stopping Conditions:** At the current node with training subset $\mathcal{S}$, check if any terminal criteria hold:
 - The node is pure ($H(\mathcal{S}) = 0$ or $G(\mathcal{S}) = 0$, meaning all labels y_i are identical)
 - The current depth equals the maximum allowed depth ($\text{depth} \geq \text{max_depth}$)
 - The number of samples is below the splitting threshold ($|\mathcal{S}| < \text{min_samples_split}$)

If any condition is satisfied, declare the node a **leaf node** ℓ and assign it a fixed class prediction $c_\ell \in \{0, 1\}$ based on the empirical majority vote within $\mathcal{S}$:

$$c_\ell = \mathbf{1}[\hat{p}(\mathcal{S}) \geq \tfrac{1}{2}] = \mathbf{1}\left[\frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} y_i \geq \tfrac{1}{2}\right].$$

If half or more of the passengers in that leaf survived ($\hat{p} \geq \frac{1}{2}$), the node predicts survival ($c_\ell = 1$), otherwise, it predicts non-survival ($c_\ell = 0$).

2. **Split Search:** Search across all features $j \in \{1, \dots, p\}$ and possible split thresholds $\tau \in \mathbb{R}$ to find the optimal split parameters (j^*, τ^*) that maximize the information gain:

$$(j^*, \tau^*) = \operatorname{argmax}_{j, \tau} \Delta(j, \tau).$$

3. **Partition:** Partition $\mathcal{S}$ into child subsets $\mathcal{S}_L = \{i \in \mathcal{S} : x_{ij^*} \leq \tau^*\}$ and $\mathcal{S}_R = \{i \in \mathcal{S} : x_{ij^*} > \tau^*\}$, and repeat the process recursively on both $\mathcal{S}_L$ and $\mathcal{S}_R$.

When the decision tree finishes growing, it divides the feature space $\mathbb{R}^p$ into L separate sections, $R_1, \dots, R_L$, corresponding to its L leaves.

¹⁵Jensen's inequality states that for any concave function f and convex weights $\lambda_1, \lambda_2 \geq 0$ summing to 1, $f(\lambda_1 x_1 + \lambda_2 x_2) \geq \lambda_1 f(x_1) + \lambda_2 f(x_2)$.

To predict whether an unseen passenger with features $x \in \mathbb{R}^p$ survives, the tree traces x down its decision rules to a single leaf region R_k and returns that region's majority vote $c_k \in \{0, 1\}$:

$$T(x) = \sum_{\ell=1}^L c_{\ell} \cdot \mathbf{1}[x \in R_{\ell}].$$

Because the leaf regions $\{R_1, \dots, R_L\}$ form a strict partition of the feature space (they are mutually exclusive and cover all possibilities), passenger x belongs to exactly one region, say R_k . Consequently, the indicator function $\mathbf{1}[x \in R_{\ell}]$ equals 1 for $\ell = k$ and 0 for all $\ell \neq k$.

Expanding the summation demonstrates this cancellation explicitly:

$$T(x) = c_1 \cdot \mathbf{1}[x \in R_1] + \dots + c_k \cdot \mathbf{1}[x \in R_k] + \dots + c_L \cdot \mathbf{1}[x \in R_L]$$

which simplifies instantly to:

$$T(x) = c_1 \cdot 0 + \dots + c_k \cdot 1 + \dots + c_L \cdot 0 = c_k.$$

Remark 3.1 (Bias-Variance Tradeoff in Single Decision Trees)

A decision tree grown without limits (where every leaf group is completely pure) gets zero training errors and has **low bias**.¹⁶ However, such deep trees suffer from **high variance**:¹⁷

- Small changes in the training data can easily change the chosen split feature and cutoff point near the top of the tree.
- Because lower branches depend on those early splits, these changes multiply down the tree, creating entirely different decision rules.

Settings such as `max_depth` (limiting tree depth) and `min_samples_split` (requiring a minimum group size to split) keep the tree simple. By accepting a slight drop in training accuracy, the model avoids memorizing noisy patterns and makes more reliable predictions on new passengers.

3.2 Random Forest: Averaging Decorrelated Trees

Single decision trees are prone to overfitting and high variance because small changes in the training data can completely change the tree's predictions. A **Random Forest** fixes this by training many trees on different random subsets of the data and restricting each split to a random subset of features. Averaging predictions across these trees lowers variance and yields far more stable predictions on unseen data.

Definition 3.4 (Bootstrap Sample)

A Random Forest builds an ensemble of N trees. To train any individual tree $b \in \{1, \dots, N\}$, we create a new training set $\mathcal{D}^{(b)}$ by drawing n samples from the original dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ at random *with replacement* (a process called bootstrapping):

$$i_1^{(b)}, \dots, i_n^{(b)} \stackrel{\text{iid}}{\sim} \text{Uniform}\{1, \dots, n\} \implies \mathcal{D}^{(b)} = \left\{ \left(x_{i_k^{(b)}}, y_{i_k^{(b)}} \right) \right\}_{k=1}^n.$$

Because each sample is returned to the pool after being picked, $\mathcal{D}^{(b)}$ contains duplicate copies of some passengers while leaving others out entirely.¹⁸

¹⁶**Bias** measures the errors caused by overly simple model assumptions. Here, low bias means a deep tree makes few assumptions and can learn complex patterns, such as interactions between age, class, and ticket type.

¹⁷**Variance** measures how much a model changes when trained on different data. Here, high variance means a tree trained on a slightly different group of passengers will pick different initial splits, changing the whole branch structure and making very different predictions on new passengers.

¹⁸The bootstrapped dataset $\mathcal{D}^{(b)}$ retains the original size (n), but random selection with replacement results in duplicate draws for some passengers and the omission of others.

Lemma 3.1 (The 63.2% Rule of Bootstrap Inclusion)

As $n \rightarrow \infty$, the probability that a specific training observation (x_i, y_i) is included in a given bootstrap sample $\mathcal{D}^{(b)}$ converges to $1 - e^{-1} \approx 0.632$.

Proof.

Consider a fixed observation i . In a single uniform draw from n items, the probability of *not* picking observation i is $1 - \frac{1}{n}$. Because the n draws are independent with replacement, the probability that observation i is omitted across all n draws is:

$$P(i \notin \mathcal{D}^{(b)}) = \left(1 - \frac{1}{n}\right)^n.$$

Taking the limit as $n \rightarrow \infty$, we obtain the limit definition of the exponential constant:

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n = e^{-1} \approx 0.368.$$

The probability of inclusion is simply the complement: $1 - e^{-1} \approx 0.632$. □

Definition 3.5 (Random Forest Ensemble)

A Random Forest consists of an ensemble of N decision trees $\{T_1, \dots, T_N\}$. Each tree T_b is constructed independently as follows:

1. Draw a bootstrap sample $\mathcal{D}^{(b)}$ of size n from the training set $\mathcal{D}$.
2. Grow a decision tree on $\mathcal{D}^{(b)}$ recursively. At each candidate split, randomly select a subset of $m_{\text{try}} = \lfloor \sqrt{p} \rfloor$ features¹⁹ out of all p features, searching for the optimal split (j^*, τ^*) within this sampled feature set.

For a test sample $x \in \mathbb{R}^p$, each individual tree $T_b(x) \in \{0, 1\}$ outputs a binary prediction. The ensemble assigns its final prediction via majority vote:

$$\hat{y}(x) = \mathbf{1} \left[\frac{1}{N} \sum_{b=1}^N T_b(x) \geq \frac{1}{2} \right].$$

¹⁹ $\lfloor \sqrt{p} \rfloor$ denotes the floor function applied to $\sqrt{p}$, rounding down to the nearest integer. For example, with $p = 10$ features, $m_{\text{try}} = \lfloor \sqrt{10} \rfloor = 3$ features are candidate evaluated at each split.

4 Model Comparison: Decision Trees, Random Forests, and Logistic Regression

Choosing between a linear model like Logistic Regression and tree-based ensemble methods depends on the underlying structure of the data and the objective of the analysis.

1. **Decision Boundaries and Interactions:** Logistic Regression constructs a linear decision boundary in the log odds space using the score $z = w^\top x + b$. It performs exceptionally well when the relationship between features and the target is approximately linear. However, it cannot capture complex, non-linear interactions (such as how passenger survival might depend on a specific combination of age, ticket class, and fare) unless interaction terms are manually engineered. In contrast, decision trees and Random Forests naturally partition the feature space into separate regions, allowing them to capture intricate non-linear patterns.
2. **Interpretability vs. Predictive Capacity:** A single decision tree is easy to interpret because its decision paths form direct, step by step logic rules. Logistic Regression is also interpretable because each weight w_j directly measures how feature x_j changes the predicted log odds. A Random Forest sacrifices this direct interpretability for higher accuracy.
3. **Sensitivity to Scaling and Outliers:** Logistic Regression is sensitive to feature scaling and outliers, making data standardization crucial. Tree-based models are unaffected by feature scaling because their splits depend only on whether one value is larger than another, not on the exact numbers. Trees are also robust to outliers because splits depend only on whether a feature value is above or below a threshold, regardless of how extreme that value is.

5 Empirical Validation on Real Data

To evaluate the comparative performance of parametric linear models versus non-parametric tree architectures, we train and validate every model configuration on the [Kaggle Titanic dataset](#) ($n_{\text{train}} = 891$ labeled training passengers, $n_{\text{test}} = 418$ unseen evaluation passengers).

Following data cleaning and missing value imputation, the dataset is split into two specialized representations:

- **Linear Model:** Qualitative predictors are encoded using $k - 1$ indicator variables with a single reference category omitted per group to avoid linear dependence (the *dummy variable trap*; see Section 3.3 of [1]). Continuous predictors (`Pclass`, `Age`, `SibSp`, `Parch`, `Fare`, `FamilySize`) are Z -score standardized and binary indicators (`WomanOrChild`, `Female`, `Title`) remain unscaled in $\{0, 1\}$.
- **Tree Model:** Tree models do not need continuous features like `Age` or `Fare` to be scaled, because splits depend only on the order of values. We can keep all category options as features without leaving any of them,²⁰ because trees test each feature independently.

Model hyperparameters are tuned before retraining on the full training manifest and generating test predictions.

Model Configuration	Train Acc.	Test Survivors ($n = 418$)	Kaggle Score
Logistic Regression (Unregularized MLE)	82.83%	168 (40.19%)	0.77511
Logistic Regression (Ridge L_2 , $\lambda = 0.0001$)	83.73%	167 (39.95%)	0.77272
Logistic Regression (Lasso L_1 , $\lambda = 1.0$)	83.39%	169 (40.43%)	0.76794
Logistic Regression (Elastic Net, $\lambda_1 = \lambda_2 = 10^{-4}$)	83.73%	167 (39.95%)	0.77272
Decision Tree (Single, depth = 5, $n_{\text{min}} = 20$)	84.96%	125 (29.90%)	0.75358
Random Forest ($N = 100$ trees, depth = 3)	83.50%	163 (38.99%)	0.77751

Key Empirical Observations

1. **High Agreement Between Models:** Despite working very differently (linear optimization vs. recursive tree splits), the Random Forest and Unregularized Logistic Regression agree on **97.85%** of all test predictions (differing on only 9 out of 418 passengers). These two models achieve the best test scores at **0.77751** (Random Forest) and **0.77511** (Logistic Regression).
2. **Random Forest Reduces Tree Variance:** A single Decision Tree predicts significantly fewer survivors on the test set, forecasting 125 survivors (29.90%) and scoring **0.75358**. Combining 100 trees using bootstrap samples and random feature subsets ($m_{\text{try}} = 5$) stabilizes predictions, raising the score to **0.77751** and predicting 163 survivors (38.99%).
3. **Feature Selection with L_1 Penalty:** All four logistic regression models make very similar predictions (167 to 169 survivors). Setting $\lambda = 1.0$ in the L_1 penalty zeros out 7 of the 27 features (25.9%): `Title_Miss`, `Title_Rare`, `Ticket_Small`, `Cabin_A`, `Cabin_B`, `Cabin_G`, and `Cabin_T`. This produces a simpler model with only a small drop in score (0.76794 vs. 0.77272).
4. **Key Survival Features:** Looking at parameter weights and split choices reveals three main patterns:

²⁰In linear models, one option is left out (e.g., dropping `Embarked_S`) to avoid collinearity. Decision trees check one feature at a time, so including all options causes no issues.

- *Women and Children First*: **WomanOrChild** ($w = +1.245$, 7.58% tree importance) and **Female** ($w = +0.977$, 5.98% tree importance) provide the largest positive boost to survival odds.
- *Socioeconomic Class*: Lower ticket class heavily reduces survival chances (**Pclass** $w = -0.763$; 8.60% forest importance), while **Fare** is the most frequently split continuous feature (10.20% importance, 70 splits).
- *Group Size Penalty*: Traveling in large groups lowers survival odds across all models (**Ticket_Large**: $w = -0.494$; **Family_Large**: $w = -0.452$), likely due to difficulties staying together during the evacuation.

Acknowledgments

The author carried out the theoretical derivations, feature engineering, and model training for this project. Anthropic’s Claude and Google’s Gemini were consulted for LaTeX layout editing, language polishing, and checking mathematical derivations.

Appendices

Appendix A Appendix: Derivation of L_1 Weight Updates

This appendix provides the full mathematical derivation for updating weights under L_1 Lasso regularization. It explains why standard gradient descent fails at zero and proves how proximal gradient descent can solve this.

A.1 The Loss Function and Derivative

Consider the Lasso regularized cross-entropy loss function:

$$J_\lambda(\theta) = \text{CE}(\theta) + \frac{\lambda}{n} \|w\|_1, \quad \text{where } \|w\|_1 = \sum_{j=1}^p |w_j|.$$

For any non-zero weight coordinate $w_j \neq 0$, the absolute value function $|w_j|$ is differentiable, giving the partial derivative:

$$\frac{\partial J_\lambda}{\partial w_j} = \frac{\partial \text{CE}}{\partial w_j} + \frac{\lambda}{n} \text{sign}(w_j) = \frac{1}{n} \left[X^\top (\hat{y} - y) \right]_j + \frac{\lambda}{n} \text{sign}(w_j).$$

Because $|w_j|$ has a non-differentiable corner at $w_j = 0$, standard gradient descent cannot evaluate $\text{sign}(0)$ to determine the next step.

A.2 Local Quadratic Approximation

To handle the non-differentiability of $|w_j|$ at zero, *proximal gradient descent* constructs a local approximation at each step. This model combines a linear expansion of the smooth loss with a quadratic penalty and the L_1 term.

Derivation of the Local Model We approximate the cross-entropy loss $\text{CE}(w)$ around $w_j^{(t)}$ using a first-order Taylor expansion:

$$\text{CE}(w) \approx \text{CE}(w_j^{(t)}) + \left. \frac{\partial \text{CE}}{\partial w_j} \right|_{\theta^{(t)}} (w - w_j^{(t)}).$$

Because a linear model has no minimum,²¹ we add a quadratic penalty $\frac{1}{2\alpha} (w - w_j^{(t)})^2$ to keep w close to $w_j^{(t)}$.

To see why the scaling factor is $\frac{1}{2\alpha}$, consider the unregularized case ($\lambda = 0$). Setting the derivative with respect to w to zero yields:

$$\frac{d}{dw} \left[\text{CE}(w_j^{(t)}) + \left. \frac{\partial \text{CE}}{\partial w_j} \right|_{\theta^{(t)}} (w - w_j^{(t)}) + \frac{1}{2\alpha} (w - w_j^{(t)})^2 \right] = 0.$$

Differentiating and solving for w (which becomes the new weight $w_j^{(t+1)}$) gives:

$$\left. \frac{\partial \text{CE}}{\partial w_j} \right|_{\theta^{(t)}} + \frac{1}{\alpha} (w - w_j^{(t)}) = 0 \implies w_j^{(t+1)} = w_j^{(t)} - \alpha \left. \frac{\partial \text{CE}}{\partial w_j} \right|_{\theta^{(t)}}.$$

The factor $\frac{1}{2\alpha}$ ensures that without regularization, this update exactly matches standard gradient descent with learning rate α .

Adding the L_1 penalty $\frac{\lambda}{n} |w|$ gives the complete minimization problem for $w_j^{(t+1)}$:

$$w_j^{(t+1)} = \arg \min_w \left\{ \text{CE}(w_j^{(t)}) + \left. \frac{\partial \text{CE}}{\partial w_j} \right|_{\theta^{(t)}} (w - w_j^{(t)}) + \frac{1}{2\alpha} (w - w_j^{(t)})^2 + \frac{\lambda}{n} |w| \right\}.$$

²¹For any non-zero derivative, the linear term $\left. \frac{\partial \text{CE}}{\partial w_j} \right|_{\theta^{(t)}} (w - w_j^{(t)})$ goes to $-\infty$ as $w \rightarrow \pm\infty$.

Completing the Square Dropping the constant term $\text{CE}(w_j^{(t)})$ and factoring out $\frac{1}{2\alpha}$ gives:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2\alpha} \left[\left(w - w_j^{(t)} \right)^2 + 2\alpha \frac{\partial \text{CE}}{\partial w_j} \Big|_{\theta^{(t)}} \left(w - w_j^{(t)} \right) \right] + \frac{\lambda}{n} |w| \right\}.$$

Adding $\alpha^2 \left(\frac{\partial \text{CE}}{\partial w_j} \Big|_{\theta^{(t)}} \right)^2$ completes the square:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2\alpha} \left(w - \left[w_j^{(t)} - \alpha \frac{\partial \text{CE}}{\partial w_j} \Big|_{\theta^{(t)}} \right] \right)^2 + \frac{\lambda}{n} |w| \right\}.$$

Let $z_j^{(t)}$ be the unregularized gradient step:

$$z_j^{(t)} = w_j^{(t)} - \alpha \frac{\partial \text{CE}}{\partial w_j} \Big|_{\theta^{(t)}} = w_j^{(t)} - \frac{\alpha}{n} \left[X^\top (\hat{y}^{(t)} - y) \right]_j.$$

Multiplying the expression inside $\arg \min_w$ by α simplifies the problem to:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2} \left(w - z_j^{(t)} \right)^2 + \frac{\alpha \lambda}{n} |w| \right\}.$$

A.3 Proof by Cases

To find the update $w_j^{(t+1)}$, we minimize the convex function:

$$g(w) = \frac{1}{2} \left(w - z_j^{(t)} \right)^2 + \frac{\alpha \lambda}{n} |w|.$$

Because $|w|$ is defined piecewise, we evaluate $g(w)$ across three cases.

1. Positive Case ($w > 0$) When $w > 0$, $|w| = w$, making $g(w)$ differentiable:

$$g(w) = \frac{1}{2} \left(w - z_j^{(t)} \right)^2 + \frac{\alpha \lambda}{n} w.$$

Setting the derivative to zero gives:

$$\frac{dg}{dw} = w - z_j^{(t)} + \frac{\alpha \lambda}{n} = 0 \implies w_j^{(t+1)} = z_j^{(t)} - \frac{\alpha \lambda}{n}.$$

For $w_j^{(t+1)} > 0$, this solution requires $z_j^{(t)} > \frac{\alpha \lambda}{n}$.

2. Negative Case ($w < 0$) When $w < 0$, $|w| = -w$, making $g(w)$ differentiable:

$$g(w) = \frac{1}{2} \left(w - z_j^{(t)} \right)^2 - \frac{\alpha \lambda}{n} w.$$

Setting the derivative to zero gives:

$$\frac{dg}{dw} = w - z_j^{(t)} - \frac{\alpha \lambda}{n} = 0 \implies w_j^{(t+1)} = z_j^{(t)} + \frac{\alpha \lambda}{n}.$$

For $w_j^{(t+1)} < 0$, this solution requires $z_j^{(t)} < -\frac{\alpha \lambda}{n}$.

3. Zero Case ($w = 0$) When $|z_j^{(t)}| \leq \frac{\alpha\lambda}{n}$, $w_j^{(t+1)} = 0$ is the minimum. Examining the derivatives on either side of zero:

- **Moving right ($w > 0$):** As $w \rightarrow 0^+$, the derivative is $\frac{\alpha\lambda}{n} - z_j^{(t)} \geq 0$, so $g(w)$ increases.
- **Moving left ($w < 0$):** As $w \rightarrow 0^-$, the derivative is $-\frac{\alpha\lambda}{n} - z_j^{(t)} \leq 0$, so $g(w)$ increases.

Because moving away from zero in either direction increases $g(w)$, and $g(w)$ is convex,²² $w_j^{(t+1)} = 0$ is the unique global minimum.

Summary Combining these three cases yields the weight update $w_j^{(t+1)}$:

$$w_j^{(t+1)} = \begin{cases} z_j^{(t)} - \frac{\alpha\lambda}{n}, & \text{if } z_j^{(t)} > \frac{\alpha\lambda}{n} \\ z_j^{(t)} + \frac{\alpha\lambda}{n}, & \text{if } z_j^{(t)} < -\frac{\alpha\lambda}{n} \\ 0, & \text{if } |z_j^{(t)}| \leq \frac{\alpha\lambda}{n} \end{cases}$$

A.4 The L_1 Update Rule

Combining the three optimal cases yields the piecewise update:

$$w_j^{(t+1)} = \begin{cases} z_j^{(t)} - \frac{\alpha\lambda}{n}, & \text{if } z_j^{(t)} > \frac{\alpha\lambda}{n} \\ z_j^{(t)} + \frac{\alpha\lambda}{n}, & \text{if } z_j^{(t)} < -\frac{\alpha\lambda}{n} \\ 0, & \text{if } |z_j^{(t)}| \leq \frac{\alpha\lambda}{n} \end{cases}$$

This update rule can be written more concisely using the $\text{sign}(\cdot)$ function and the $\max(0, \cdot)$ operator:

$$w_j^{(t+1)} = \text{sign}\left(z_j^{(t)}\right) \max\left(0, |z_j^{(t)}| - \frac{\alpha\lambda}{n}\right).$$

Substituting the unregularized step $z_j^{(t)} = w_j^{(t)} - \frac{\alpha}{n} [X^\top(\hat{y}^{(t)} - y)]_j$ gives the explicit weight update:

$$w_j^{(t+1)} = \text{sign}\left(w_j^{(t)} - \frac{\alpha}{n} [X^\top(\hat{y}^{(t)} - y)]_j\right) \max\left(0, \left|w_j^{(t)} - \frac{\alpha}{n} [X^\top(\hat{y}^{(t)} - y)]_j\right| - \frac{\alpha\lambda}{n}\right).$$

This completes the proof that L_1 regularization produces exact zero weights whenever the unregularized step size is less than or equal to $\frac{\alpha\lambda}{n}$.

²²For any convex function, a local minimum is guaranteed to be the unique global minimum.

Appendix B Appendix: Derivation of Elastic Net Updates and Behavior of Correlated Features

This appendix provides the step-by-step mathematical derivation for updating parameters under Elastic Net regularization.

B.1 The Loss Function and Derivative

Consider the Elastic Net regularized cross-entropy loss function:

$$J_{\lambda_1, \lambda_2}(\theta) = \text{CE}(\theta) + \frac{\lambda_1}{n} \|w\|_1 + \frac{\lambda_2}{2n} \|w\|_2^2, \quad \text{where } \|w\|_1 = \sum_{j=1}^p |w_j| \text{ and } \|w\|_2^2 = \sum_{j=1}^p w_j^2.$$

For any non-zero weight coordinate $w_j \neq 0$, the partial derivative with respect to w_j incorporates both penalty components:

$$\frac{\partial J_{\lambda_1, \lambda_2}}{\partial w_j} = \frac{1}{n} \left[X^\top (\hat{y} - y) \right]_j + \frac{\lambda_1}{n} \text{sign}(w_j) + \frac{\lambda_2}{n} w_j.$$

Because the L_1 term is non-differentiable at $w_j = 0$, standard gradient descent cannot evaluate the derivative at zero. We address this using proximal coordinate descent.

B.2 Local Quadratic Approximation for Elastic Net

Following the quadratic model established in Appendix A, we construct a local approximation around $w_j^{(t)}$ and add both regularization penalties:

$$w_j^{(t+1)} = \arg \min_w \left\{ \text{CE} \left(w_j^{(t)} \right) + \frac{\partial \text{CE}}{\partial w_j} \Big|_{\theta^{(t)}} \left(w - w_j^{(t)} \right) + \frac{1}{2\alpha} \left(w - w_j^{(t)} \right)^2 + \frac{\lambda_1}{n} |w| + \frac{\lambda_2}{2n} w^2 \right\}.$$

Combining Quadratic Terms Dropping the constant term $\text{CE} \left(w_j^{(t)} \right)$ and completing the square for the loss terms using the unregularized step $z_j^{(t)} = w_j^{(t)} - \frac{\alpha}{n} \left[X^\top (\hat{y}^{(t)} - y) \right]_j$ yields:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2\alpha} \left(w - z_j^{(t)} \right)^2 + \frac{\lambda_1}{n} |w| + \frac{\lambda_2}{2n} w^2 \right\}.$$

Multiplying the expression inside $\arg \min_w$ by α simplifies the objective to:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2} \left(w - z_j^{(t)} \right)^2 + \frac{\alpha \lambda_1}{n} |w| + \frac{\alpha \lambda_2}{2n} w^2 \right\}.$$

Expanding the squared term $\frac{1}{2} \left(w - z_j^{(t)} \right)^2 = \frac{1}{2} w^2 - z_j^{(t)} w + \frac{1}{2} \left(z_j^{(t)} \right)^2$ and dropping the constant $\frac{1}{2} \left(z_j^{(t)} \right)^2$ gives:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2} w^2 - z_j^{(t)} w + \frac{\alpha \lambda_1}{n} |w| + \frac{\alpha \lambda_2}{2n} w^2 \right\}.$$

Combining the terms involving w^2 yields:

$$w_j^{(t+1)} = \arg \min_w \left\{ \frac{1}{2} \left(1 + \frac{\alpha \lambda_2}{n} \right) w^2 - z_j^{(t)} w + \frac{\alpha \lambda_1}{n} |w| \right\}.$$

B.3 Proof by Cases

To find $w_j^{(t+1)}$, we minimize $h(w) = \frac{1}{2} \left(1 + \frac{\alpha \lambda_2}{n} \right) w^2 - z_j^{(t)} w + \frac{\alpha \lambda_1}{n} |w|$ across three cases:

1. Positive Case ($w > 0$) When $w > 0$, $|w| = w$. Setting the derivative to zero gives:

$$\frac{dh}{dw} = \left(1 + \frac{\alpha\lambda_2}{n}\right) w - z_j^{(t)} + \frac{\alpha\lambda_1}{n} = 0 \implies w_j^{(t+1)} = \frac{z_j^{(t)} - \frac{\alpha\lambda_1}{n}}{1 + \frac{\alpha\lambda_2}{n}}.$$

For $w_j^{(t+1)} > 0$, this solution requires $z_j^{(t)} > \frac{\alpha\lambda_1}{n}$.

2. Negative Case ($w < 0$) When $w < 0$, $|w| = -w$. Setting the derivative to zero gives:

$$\frac{dh}{dw} = \left(1 + \frac{\alpha\lambda_2}{n}\right) w - z_j^{(t)} - \frac{\alpha\lambda_1}{n} = 0 \implies w_j^{(t+1)} = \frac{z_j^{(t)} + \frac{\alpha\lambda_1}{n}}{1 + \frac{\alpha\lambda_2}{n}}.$$

For $w_j^{(t+1)} < 0$, this solution requires $z_j^{(t)} < -\frac{\alpha\lambda_1}{n}$.

3. Zero Case ($w = 0$) When $|z_j^{(t)}| \leq \frac{\alpha\lambda_1}{n}$, $w_j^{(t+1)} = 0$ is the minimum because moving away from zero in either direction increases $h(w)$.

Unified Update Expression Combining these cases yields the explicit Elastic Net coordinate update:

$$w_j^{(t+1)} = \frac{\text{sign}\left(z_j^{(t)}\right) \max\left(0, \left|z_j^{(t)}\right| - \frac{\alpha\lambda_1}{n}\right)}{1 + \frac{\alpha\lambda_2}{n}}.$$

Substituting the unregularized step $z_j^{(t)} = w_j^{(t)} - \frac{\alpha}{n} [X^\top(\hat{y}^{(t)} - y)]_j$ gives:

$$w_j^{(t+1)} = \frac{\text{sign}\left(w_j^{(t)} - \frac{\alpha}{n} [X^\top(\hat{y}^{(t)} - y)]_j\right) \max\left(0, \left|w_j^{(t)} - \frac{\alpha}{n} [X^\top(\hat{y}^{(t)} - y)]_j\right| - \frac{\alpha\lambda_1}{n}\right)}{1 + \frac{\alpha\lambda_2}{n}}.$$

B.4 Behavior of Correlated Features

To analyze how regularization behaves with correlated features, consider two identical predictors $X_1 = X_2 = x$. The model output is:

$$\hat{y} = w_1 X_1 + w_2 X_2 = (w_1 + w_2)x.$$

Because predictions depend only on the sum $S = w_1 + w_2$, any weight pair satisfying $w_1 + w_2 = S$ produces the exact same cross-entropy loss $\text{CE}(S)$.

L_1 Instability Consider first the case of L_1 regularization alone ($\lambda_2 = 0$). For non-negative weights ($w_1, w_2 \geq 0$), any pair satisfying $w_1 + w_2 = S$ gives the L_1 penalty:

$$P_{L_1}(w_1, w_2) = \|w\|_1 = w_1 + w_2 = S. \quad (8)$$

The total loss becomes:

$$J_{\lambda_1}(w_1, w_2) = \text{CE}(w_1 + w_2) + \frac{\lambda_1}{n} P_{L_1}(w_1, w_2) = \text{CE}(S) + \frac{\lambda_1}{n} S.$$

Because this value depends only on the optimized S value, the loss is constant for any choice of w_1 and w_2 that sums to S . Every pair from $(S, 0)$ to $(0, S)$ gives the exact same loss, creating infinitely many minimizers. Consequently, optimization algorithms select a final solution based entirely on their initial starting values $w_1^{(0)}$ and $w_2^{(0)}$.²³

²³Applying the chain rule to $J(w_1, w_2) = \text{CE}(w_1 + w_2) + \frac{\lambda_1}{n}(w_1 + w_2)$ gives:

$$\frac{\partial J}{\partial w_1} = \text{CE}'(w_1 + w_2) \cdot \frac{\partial(w_1 + w_2)}{\partial w_1} + \frac{\lambda_1}{n} \cdot \frac{\partial(w_1 + w_2)}{\partial w_1} = \text{CE}'(w_1 + w_2)(1) + \frac{\lambda_1}{n}(1).$$

L_2 Unique Solution Adding the L_2 penalty forces a unique choice. For any pair where $w_1 + w_2 = S$, the total Elastic Net loss is:

$$J_{\lambda_1, \lambda_2}(w_1, w_2) = \text{CE}(w_1 + w_2) + \frac{\lambda_1}{n}(w_1 + w_2) + \frac{\lambda_2}{2n}(w_1^2 + w_2^2).$$

Substituting $w_2 = S - w_1$ gives:

$$J_{\lambda_1, \lambda_2}(w_1) = \text{CE}(S) + \frac{\lambda_1}{n}S + \frac{\lambda_2}{2n}[2w_1^2 - 2Sw_1 + S^2].$$

Because $\text{CE}(S)$ and $\frac{\lambda_1}{n}S$ are fixed for a given sum S , setting the derivative with respect to w_1 to zero yields a single minimizer for $w_j^{(t+1)}$:

$$\frac{dJ_{\lambda_1, \lambda_2}}{dw_1} = \frac{\lambda_2}{2n}(4w_1 - 2S) = 0 \implies w_1^{(t+1)} = \frac{S}{2}, \quad w_2^{(t+1)} = \frac{S}{2}.$$

Introducing the L_2 penalty removes the non-uniqueness issue and guarantees that the updated weights $(w_1^{(t+1)}, w_2^{(t+1)})$ no longer depend on initial conditions $w_j^{(0)}$.

References

- [1] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning: with Applications in Python*. Springer Texts in Statistics. Springer, 2023.

Because $\frac{\partial(w_1+w_2)}{\partial w_1} = \frac{\partial(w_1+w_2)}{\partial w_2} = 1$, the partial derivatives are identical ($\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial w_2}$), so $\nabla J = \frac{\partial J}{\partial w_1} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Under gradient descent:

$$\begin{bmatrix} w_1^{(t+1)} \\ w_2^{(t+1)} \end{bmatrix} = \begin{bmatrix} w_1^{(t)} \\ w_2^{(t)} \end{bmatrix} - \alpha \nabla J(w_1^{(t)}, w_2^{(t)}) \implies w_1^{(t+1)} - w_2^{(t+1)} = w_1^{(t)} - w_2^{(t)}.$$

The difference $w_1^{(t)} - w_2^{(t)}$ stays the same across every step t . Combining $w_1^* - w_2^* = w_1^{(0)} - w_2^{(0)}$ with $w_1^* + w_2^* = S$ gives the final solution $w_1^* = \frac{S + (w_1^{(0)} - w_2^{(0)})}{2}$ and $w_2^* = \frac{S - (w_1^{(0)} - w_2^{(0)})}{2}$.